Machine Learning Assignment

1. Do you think both Convolution Neural Network (CNN) and Recurrent Neural Network (RNN) are required for this sign word recognition task? Justify your answer.

Yes, both CNN and RNN are crucial for this task, and here is my justification:

- **CNN (Convolutional Neural Network)**:
    - The BdSLW60 dataset consists of video frames where each frame represents a static image of a hand gesture. CNN is designed to capture spatial features from each image. The convolutional layers identify key patterns such as hand shape, finger positioning, and orientation of the hand. These features are essential for recognizing the specific hand gestures.
    - CNNs help in extracting hierarchical features, starting from low-level features (like edges) and progressing to high-level features (like hand shapes and patterns). Pooling layers reduce spatial dimensions, making the system more efficient and robust to small translations or distortions in the hand gesture.
- **RNN (Recurrent Neural Network)**:
    - Sign language gestures are not just static but involve movement and changes over time, making it a dynamic task. An RNN is well-suited to handle temporal dependencies like how the gesture evolves over multiple frames in a sequence. This sequential aspect is crucial because hand gestures often have a beginning, middle, and end, and their recognition depends on the transition from one gesture to the next.
    - Specifically, LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Units) are effective in capturing long-term dependencies and avoiding the vanishing gradient problem, which is common in traditional RNNs. By using RNNs, the model can process the sequence of frames, remembering key features of earlier frames, and use that information to interpret the meaning of the entire gesture.

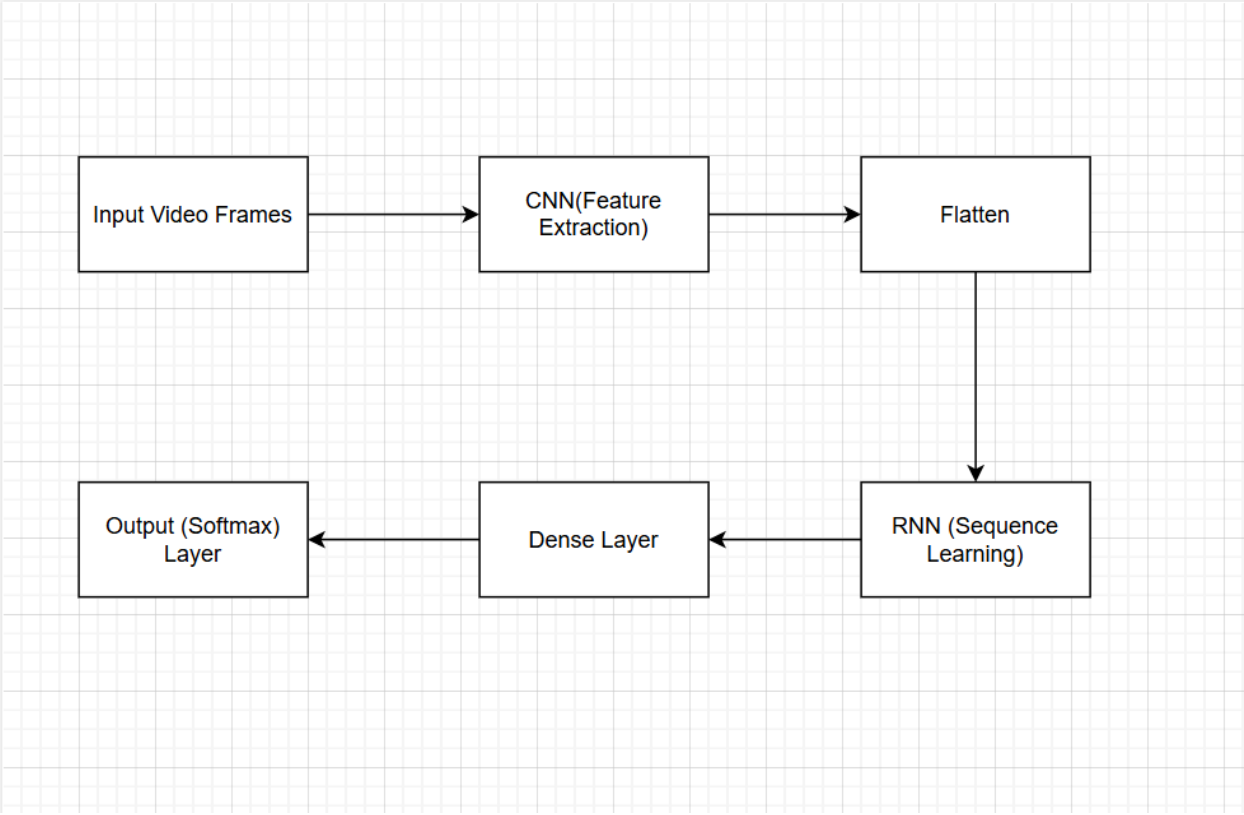
Thus, I think combining CNN for spatial feature extraction and RNN for temporal sequence processing allows the model to recognize both the shape and movement of the hand, making it robust and accurate for BdSL word recognition.

2. Propose a suitable deep learning-based architecture by drawing the block diagram of your proposed solution.

The proposed architecture integrates both CNN and RNN, which are necessary for analyzing both spatial and temporal features of the hand gestures. Here's the flow of the architecture:

- **Input**: Video frames representing hand gestures.
- **CNN Block**: Extracts spatial features (hand shape, orientation, etc.) from each frame.
- **Flatten Layer**: Converts the feature maps from the CNN block into a 1D array.
- **RNN Block**: Processes the sequential data (frames) to learn the temporal dynamics of the hand movements.
- **Dense Layer**: Fully connects the output from the RNN and prepares it for classification.
- **Output Layer**: A softmax layer for classifying the gesture into one of the 60 sign words.

The model processes each video frame individually (via CNN) and sequentially (via RNN) to recognize the sign gesture. Below I am giving the architecture drawn using draw.io



3. Explain how video frames and sign movement information can be processed through your model architecture.

The model processes both spatial and temporal information from the video frames using two key components:

- **Video Frames (Spatial Information)**:
    - Each frame in the video provides a static image of the hand gesture. The CNN is applied to each frame individually to extract spatial features such as the hand's shape, finger positions, and the orientation of the hand. The CNN uses convolution filters to detect low-level patterns like edges and textures, which help identify the hand's shape.
    - The CNN captures these features by applying multiple layers of convolutions and pooling, reducing the spatial dimensions and allowing the network to focus on the most relevant features for sign language recognition.
- **Sign Movement (Temporal Information)**:
    - Sign language involves dynamic gestures, meaning the hand's position and shape change over time. To capture this movement, the sequence of frames (representing different time steps in the gesture) is processed by the RNN, which is specifically designed for sequential data.
    - The RNN (LSTM/GRU) analyzes the sequence of CNN-extracted features across the frames to learn how the hand moves and transitions from one pose to another. The RNN maintains a hidden state that holds information from previous frames, allowing it to understand the temporal dynamics of the gesture. This is crucial for recognizing gestures that evolve over time, such as signs that start with a fist and gradually open into a hand or signs that involve specific hand movements.

Together, CNN handles the spatial features (hand shape and position) from each individual frame, and RNN processes the temporal features (movement and sequence of frames) to capture the motion dynamics. This combined approach enables the model to accurately recognize both static hand shapes and dynamic hand movements, which are both essential for BdSL word recognition.

4. Provide relevant mathematical formulations for CNN/RNN operations.

1. **CNN Operations**: A convolution operation in a CNN can be represented as:

$y(t) = (x * w)(t) + b$

where:

- x is the input image (or previous layer's output),
- w is the kernel (filter),
- b is the bias term,
- $*$ denotes the convolution operation,
- y(t) is the output feature map.

- After the convolution operation, an activation function like ReLU is applied:

$ReLU(x)=max(0,x)$

Pooling operations (such as MaxPooling) reduce the spatial dimensions of the feature map:

$ypool(i,j)=max\{x(k,l) \mid k, l \in window\}$

2. **RNN Operations** (specifically LSTM):

- **Forget gate**:

  $f(t)=\sigma(Wf \cdot [h(t-1),x(t)]+bf)$

- **Input gate**:

  $i(t)=\sigma(Wi \cdot [h(t-1),x(t)]+bi)$

- **Cell state**:

$$c(t) = f(t) * c(t-1) + i(t) * \tanh(W_c \cdot [h(t-1), x(t)] + b_c)$$

- **Output gate**:

$$o(t) = \sigma(W_o \cdot [h(t-1), x(t)] + b_o)$$

- **Hidden state**:

$$h(t) = o(t) * \tanh(c(t))$$

Where $x(t)$ is the input at time step t, and $h(t-1)$ is the hidden state at the previous time step.

5. Describe how you are going to evaluate your model's performance. Justify your choice on evaluation metric.

To evaluate the model's performance, I will use the following metrics :

- **Accuracy**: This is the primary metric for classification tasks. It will measure how many times the predicted sign word matches the actual word. Since there are 60 classes, accuracy is a straightforward and intuitive metric for overall performance.

- **Confusion Matrix**: This will provide a detailed breakdown of correct and incorrect classifications across all classes. It helps in identifying which specific signs are often misclassified, providing insights for improving the model.

- **F1 Score**: This is particularly useful when dealing with class imbalance. The F1 score balances precision and recall, providing a single metric that combines both aspects of classification. If certain sign words are underrepresented, F1 score will give a better understanding of the model's performance on these classes.

- **Precision & Recall**: These will be useful for evaluating how well the model is identifying specific sign words, particularly those that may be difficult to differentiate from others due to similar hand shapes or movement patterns.

Justification: Accuracy is the main metric as it provides a general sense of how well the model is classifying the signs. The confusion matrix, F1 score, precision, and recall will help in understanding where the model is making mistakes and provide more specific insights for improvement, especially if there is class imbalance in the dataset.